

STUDENT GUIDE · 45 MIN

Building Your First **AI Agent**

*A practical guide — folder structure, prompts,
tools, and a working agent loop in Python.*

Python 3.10+

Anthropic SDK

Beginner Friendly

01

What is an AI Agent?

From chatbots to autonomous loops

The core idea

An AI agent uses a language model not just to answer questions, but to reason, plan, and take actions.

It calls tools, reads files, runs code — then loops until the goal is complete.



Goal-directed

Given a task, it decides what steps to take — not just one reply.



Iterative

Loops until done: think → act → observe → repeat.



Tool-using

Can call functions your code provides (search, calculate, etc.).



Stateful

Keeps a conversation history to remember what happened.

Agent vs. Chatbot



Chatbot

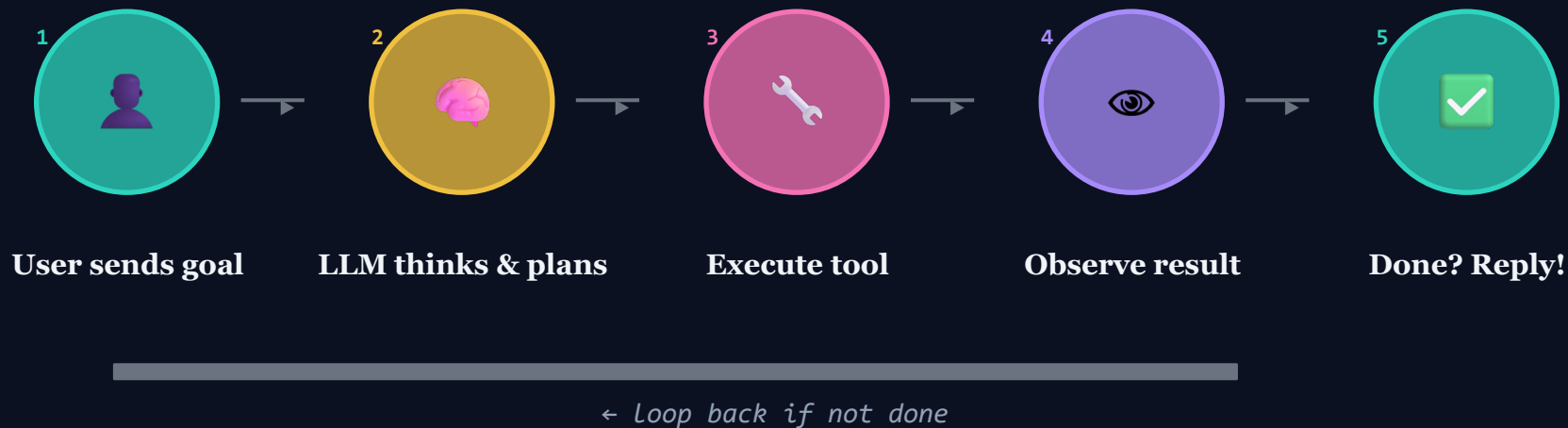
- Responds to one message at a time
- No tools — can't take actions
- Stateless (no memory by default)
- Stops after one reply
- Best for: Q&A, conversation, writing



AI Agent

- Pursues a multi-step goal
- Calls tools: search, code, files...
- Maintains conversation history
- Loops until task is complete
- Best for: research, automation, tasks

The agent loop



Key insight: the model never directly executes code. It outputs a JSON tool-call description; your Python code reads that, runs the function, and returns the result. This keeps you in control.

02

Project Structure

Where every file belongs and why

Folder structure

my_agent/

├── prompts/

│ ├── system.txt

│ └── examples/

├── tools/

│ ├── __init__.py

│ ├── calculator.py

│ └── web_search.py

└── memory/

prompts/

All system instructions and few-shot examples. Edit prompts here without touching Python.

tools/

One file per tool — schema + handler. `__init__.py` exports everything the agent needs.

memory/

Conversation history management. Lets you persist sessions across runs.

agent.py

The core while-loop. This is where the magic happens.

.env

Your API key. Add to `.gitignore` immediately — never commit secrets!

03

Managing Prompts

Keep instructions in text files, not in code

Advanced Planning Strategies

Chain-of-Thought (CoT)

Prompt the model to reason step-by-step before answering. Dramatically improves accuracy on complex tasks.

When: Multi-step math, logic, experimental design

Tree of Thoughts (ToT)

Explore multiple reasoning branches simultaneously. Evaluate and backtrack. Best for open-ended problems with many valid paths.

When: Hypothesis generation, drug design, optimization

Plan-and-Execute

First generate a full plan, then execute each step. The planner and executor can be separate agents (or models).

When: Long experiments, multi-day workflows, batch jobs

Reflexion / Self-Critique

After acting, the agent critiques its own output and retries. Verbal reinforcement learning without gradient updates.

When: Code writing, data analysis, literature synthesis



For science: combine CoT + Reflexion — reason carefully, then verify against data before committing

Why text files, not hardcoded strings

Easy editing

Change your agent's behaviour without touching any Python. Open the .txt, save, re-run.

Swappable

Run the same agent with a 'strict' or 'creative' system prompt just by pointing to a different file.

Version control

Git tracks prompt changes separately from code changes. Great for experiments.

Non-coders can help

Teammates who don't write Python can still refine prompts in a text editor.

The system prompt & loader

```
# prompts/system.txt
You are a helpful research assistant.
Always use web_search before answering factual questions.
Show your reasoning. Be concise (< 3 sentences).

# config.py
from pathlib import Path
import os
from dotenv import load_dotenv

load_dotenv()

PROMPTS_DIR = Path("prompts")
MODEL        = "claude-sonnet-4-20250514"
API_KEY      = os.getenv("ANTHROPIC_API_KEY")

def load_prompt(name: str) -> str:
    """Load a prompt file by name."""
    return (PROMPTS_DIR / name).read_text(encoding="utf-8").strip()
```



Use `prompt.format(user_name=name)` to inject dynamic values into your prompts.

04

Defining Tools

Schemas the model reads, handlers your code runs

Every tool has two parts



JSON Schema

Sent to the LLM so it knows the tool exists.

- Tool name
- Human-readable description
- Input parameters with types
- Required vs. optional fields

The schema is a declaration. The LLM reads it, decides when to use the tool, and fills in the inputs.



Python Handler

The function your code runs when the tool is called.

- Named run() by convention
- Takes the tool's inputs as arguments
- Returns a string result
- Errors should return a string too

The handler is execution. Your code maps tool names to functions and calls them with the model's inputs.

Tool example — calculator.py

```
# Schema — sent to the LLM
SCHEMA = {
    "name": "calculator",
    "description": "Evaluate a safe math expression. Returns a number.",
    "input_schema": { "type": "object",
        "properties": { "expression": {
            "type": "string" }},
        "required": ["expression"] }}

# Handler — your Python code runs this
def run(expression: str) -> str:
    try:
        tree = ast.parse(expression, mode="eval")
        result = eval(ast.compile(tree, "<expr>", "eval"))
        return str(result)
    except Exception as e:
        return f"Error: {e}"
```



Use `ast.parse` to safely evaluate math — never use `eval()` on raw user input directly.

Registering all tools — `__init__.py`

```
from . import calculator, web_search, file_reader

# Schemas → passed to the API as tools=
ALL_SCHEMAS = [
    calculator.SCHEMA,
    web_search.SCHEMA,
    file_reader.SCHEMA,
]

# Handlers → map name to function
HANDLERS = {
    "calculator" : calculator.run,
    "web_search" : web_search.run,
    "file_reader": file_reader.run,
}
```



Add a new tool in 3 steps: create the .py file, import it here, add to ALL_SCHEMAS and HANDLERS.

05

The Agent Loop

The while-loop that drives your agent

agent.py — the core loop

```
def run_agent(user_message: str) -> str:
    system = load_prompt("system.txt")
    history = [{ "role": "user", "content": user_message }]

    while True:
        response = client.messages.create(
            model=MODEL, max_tokens=MAX_TOKENS,
            system=system, tools=ALL_SCHEMAS, messages=history)

        history.append({ "role": "assistant", "content": response.content })

        if response.stop_reason == "end_turn":
            return next(b.text for b in response.content if b.type=="text")

        # Tool calls → run each, feed results back
        tool_results = []
        for block in response.content:
            if block.type != "tool_use": continue
            result = HANDLERS[block.name](**block.input)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": result})
```



Tool results go back as a "user" role message — that is how the Anthropic API works.

What happens each iteration

1



Send full history to API

Every call includes system prompt + complete message history. The model has no memory of its own.

2



Model decides: done or tool?

If stop_reason='end_turn', it wrote a final reply. If stop_reason='tool_use', it wants to call a tool.

3



You run the tool

Your Python code executes the function with the model's inputs and gets a string result back.

4



Append result, loop again

The result is added as a 'user' message. Call the API again — the model now knows what the tool returned.

06

Memory & Running

Persisting history and launching the agent

Memory Systems in AI Agents

In-Context (Working Memory)

The LLM's active context window

Holds current conversation + retrieved chunks

Limited: 8K–200K tokens depending on model

Fast but ephemeral — lost after session ends

→ Use for: active reasoning, tool results

External (Long-Term Memory)

Vector databases (Chroma, Pinecone, Weaviate)

Semantic similarity search (embeddings)

Persistent across sessions and experiments

Scales to millions of documents

→ Use for: past experiments, knowledge base

Episodic (Event Log)

Structured log of past agent actions

What was tried, what succeeded/failed

Enables learning from experience

Critical for reproducibility in science

→ Use for: avoiding repeated mistakes

Saving conversation history

```
import json
from pathlib import Path

STORE = Path("memory/store.json")

def load_history(session_id: str) -> list:
    if not STORE.exists(): return []
    return json.loads(STORE.read_text()).get(session_id, [])

def save_history(session_id: str, history: list):
    data = json.loads(STORE.read_text()) if STORE.exists() else {}
    data[session_id] = history
    STORE.write_text(json.dumps(data, indent=2))
```



For production, swap the JSON file for a database (SQLite or Redis) so history scales.

R U N N I N G

main.py — run it!

```
# main.py
from agent import run_agent

if __name__ == "__main__":
    print("🤖 Agent ready. Type 'quit' to exit.\n")
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() in {"quit", "exit"}: break
        answer = run_agent(user_input)
        print(f"\nAgent: {answer}\n")

# — terminal —
pip install anthropic python-dotenv
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
python main.py
```



Get your API key free (with limits) at console.anthropic.com — no credit card needed to start.

Where to go from here

Add web search

Tavily or Brave Search API — 1 000 free calls/mo. Your agent can look up any topic in real-time.

Vector memory

Embed past conversations with sentence-transformers and retrieve relevant snippets as context.

Sub-agents

A planner agent that breaks big goals into sub-tasks and delegates each to a specialised worker agent.

Add a UI

Wrap `run_agent()` in a FastAPI endpoint + a React/HTML frontend for a full web app.

Evals & tracing

Use LangSmith or Braintrust to log every trace, measure accuracy, and compare prompt versions.

Agent frameworks

LangGraph, AutoGen, CrewAI — explore multi-agent orchestration once your single agent works.

5 things to remember

- 01 An agent is a while-loop — it keeps calling the LLM until the job is done.
- 02 Separate prompts from code. Store them as .txt files in a prompts/ folder.
- 03 Every tool has two parts: a JSON schema (what the LLM sees) and a Python handler (what your code runs).
- 04 Tool results go back as a 'user' message — the model then gets another turn to reason.
- 05 Never commit .env. Add new tools in 3 steps: create the file, add to ALL_SCHEMAS, add to HANDLERS.

07

Level Up Your Agent

Prompt engineering, RAG, fine-tuning, frameworks & more

Advanced prompt engineering

Chain-of-Thought (CoT)

Forces the model to reason before concluding. Dramatically improves accuracy on math, logic, and multi-step tasks.

```
system.txt:  
"Think step by step before answering.  
Write your reasoning inside <thinking> tags,  
then give your final answer after."
```

Few-Shot Examples

Show the model 2–3 ideal examples of tool use. Load them into history before the user message.

```
prompts/examples/tool_use.json:  
[{"role": "user", "content": "What is 2^10?"},  
 {"role": "assistant", "content": null,  
  "tool_calls": [{"name": "calculator", ...}]]
```

Structured Output

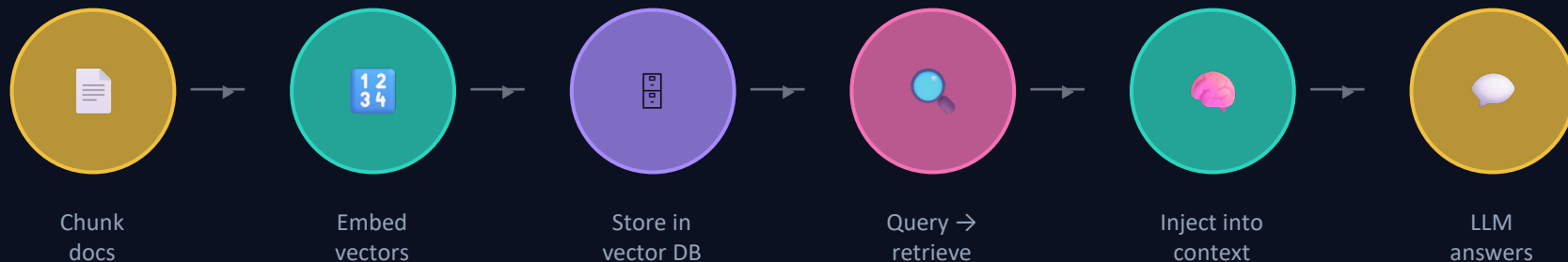
Ask for JSON output and parse it with `json.loads()`. Gives you machine-readable answers you can route on.

```
system.txt:  
"Always respond in this JSON format:  
{\"action\": \"..\",  
  \"reasoning\": \"..\",  
  \"confidence\": 0.0-1.0}"
```

RAG — Retrieval Augmented Generation

The problem: LLMs have a fixed context window. You can't paste 1,000 documents in your system prompt.

The solution: store documents as embeddings, retrieve only the most relevant chunks at query time.



```
# pip install chromadb sentence-transformers
import chromadb
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.Client() # store chunks as embeddings
def retrieve(query, n=3):
    results = collection.query(query_embeddings=[model.encode(query)], n_results=n)
    return "\n".join(results["documents"][0])
```

LangChain & LangGraph



LangChain

Batteries-included agent toolkit

- ✓ Huge ecosystem of pre-built tools
- ✓ Prompt templates & output parsers
- ✓ Easy model switching (Anthropic, OpenAI...)
- ✓ Built-in retrieval (RAG) pipelines

- ✗ Can feel like magic — hard to debug
- ✗ Heavy dependency overhead
- ✗ Abstraction may hide important details

Best for: rapid prototyping, standard RAG pipelines, switching LLM providers



LangGraph

State-machine orchestration for complex agents

- ✓ Explicit graph of nodes & edges
- ✓ Built-in persistence & checkpointing
- ✓ Human-in-the-loop support
- ✓ Best for multi-agent coordination

- ✗ Steeper learning curve than LangChain
- ✗ More boilerplate for simple agents
- ✗ Graph design requires upfront planning

Best for: multi-step pipelines, branching logic, long-running autonomous agents

LangGraph — a minimal agent graph

```
# pip install langgraph langchain-anthropic
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-20250514").bind_tools(tools)

def call_model(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

def should_continue(state) -> str:
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

graph = StateGraph(dict)
graph.add_node("agent", call_model)
graph.add_node("tools", tool_node)
graph.add_conditional_edges("agent", should_continue)
graph.add_edge("tools", "agent")
app = graph.compile(checkpointer=MemorySaver())
```



LangGraph makes the loop an explicit graph — each node is a function, edges define routing logic.

When & how to fine-tune

Don't fine-tune first

Try prompt engineering, few-shot examples, and RAG before fine-tuning. Fine-tuning is expensive, slow to iterate, and rarely the right first step.

Good reasons to fine-tune

- You have 500+ high-quality examples
- You need a very specific output format
- Latency matters — smaller fine-tuned model beats large prompted model
- Proprietary style / domain that prompts can't capture

Full Fine-Tuning

Cost: \$\$\$\$

Update all model weights on your dataset. Highest quality, but requires significant GPU compute and time. Best for large-scale domain adaptation.

LoRA / QLoRA

Cost: \$\$

Train small adapter matrices. Freeze base weights — only 0.1–1% of params are updated. Same quality at a fraction of the compute. Standard choice.

Anthropic Fine-Tuning API

Cost: \$

Upload your JSONL dataset to Anthropic, specify base model + epochs. No GPU needed. Simplest path for Claude-based agents.

Fine-tuning — preparing your JSONL dataset

```
# Every row = one training example in JSONL format
# dataset.jsonl
{"messages": [
  {"role": "system",    "content": "You are a concise code reviewer."},
  {"role": "user",      "content": "Review this Python function: def add(a,b): return a+b"},
  {"role": "assistant", "content": "Clean and correct. Add type hints for clarity: def add(a: int, b: int)
-> int."}
]}

# Python helper to build dataset from your logs
import json

def make_example(system, user, ideal_answer):
    return {"messages": [
        {"role": "system",    "content": system},
        {"role": "user",      "content": user},
        {"role": "assistant", "content": ideal_answer}
    ]}

# Write to file
```

💡 Aim for 500–5000 diverse, high-quality examples. Garbage in = garbage out. Quality beats quantity.

Multi-agent patterns

Orchestrator + Workers

A planner agent breaks the goal into subtasks, then delegates each to a specialised worker (researcher, coder, writer). Classic and reliable.

 [LangGraph](#) · [AutoGen](#) · [manual](#)

Reflection Loop

Agent A produces output. Agent B (critic) reviews it. Agent A revises. Repeat N times. Dramatic quality improvement for writing and code generation.

 [AutoGen](#) · [any two-agent setup](#)

Parallel Fan-out

Run multiple agents simultaneously on sub-problems, then merge results. Cuts wall-clock time for independent tasks like web research across sources.

 [asyncio](#) + [any agent](#) · [LangGraph branches](#)

Guardrail Agent

A fast, cheap model screens input/output before expensive processing. Catches harmful requests, validates schema, classifies intent.

 [Claude Haiku as guard](#) · [LlamaGuard](#)

Evaluation & observability

"You can't improve what you don't measure." — Every ML engineer ever.



Tracing

- Log every LLM call: input, output, latency, token cost
- Track tool calls & their results
- Group calls into a single trace per user request

🔧 LangSmith, Langfuse,
Braintrust, Arize



Automated Evals

- Write test cases: input → expected output
- Use LLM-as-judge to score responses
- Run evals on every code change in CI

🔧 Braintrust, PromptFoo, RAGAS
(for RAG)



A/B Prompt Testing

- Run prompt variant A vs B on same inputs
- Compare accuracy, coherence, cost
- Gradually roll out winning prompt version

🔧 LangSmith experiments • Statsig
• manual

Cost & performance at scale

01

Model tiering

Use Claude Haiku for classification, routing, and simple tool calls. Reserve Sonnet/Opus for complex reasoning. 10–50× cost difference.

02

Prompt caching

Anthropic's prompt caching reduces cost by up to 90% and latency by 85% when the system prompt is unchanged across requests. Add `cache_control` breakpoints.

03

Context compression

Summarise old conversation turns before they push you over the context limit. Keep the last N messages verbatim, summarise the rest into 1 paragraph.

04

Streaming responses

Use `client.messages.stream()` to start rendering output before the full response arrives. Perceived latency drops dramatically for users.

05

Batch processing

Use the Anthropic Batch API for offline tasks. Up to 50% cost reduction with no real-time SLA needed. Ideal for evals and bulk document processing.

Your improvement roadmap

Phase 1

Days 1–7

✓ Working agent loop
(today)

✓ Prompts in .txt files

✓ 2–3 tools registered

✓ Runs from terminal

Phase 2

Week 2–3

Add web search + RAG
(ChromaDB)

Few-shot examples in
prompts/

Structured JSON output
parsing

LangSmith tracing
connected

Phase 3

Month 2

Migrate loop to LangGraph

Add reflection / critic agent

Automated eval suite (10+
tests)

Streaming + FastAPI web
endpoint

Phase 4

Month 3+

Fine-tune on your logged
examples

Multi-agent orchestration

Model tiering (Haiku guards)

Prompt caching + Batch API



Questions?

`docs.anthropic.com` · `github.com/anthropics` · `console.anthropic.com`